

Unsafe by Flow: Uncovering Bidirectional Data-Flow Risks in MCP Ecosystem

Xinyi Hou
xinyihou@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Yanjie Zhao
yanjie_zhao@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Haoyu Wang*
haoyuwang@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Abstract

Model Context Protocol (MCP) have quickly become the interface layer between LLM agents and external tools, yet they also introduce unsafe data flows that existing analyzers handle poorly. Vulnerabilities manifest in two directions: requester-controlled arguments may propagate to sensitive operations, while untrusted external or sensitive internal data may surface through MCP-visible outputs and subsequently influence host or model behavior. Accurate detection is complicated by the heterogeneous registration and dispatch patterns MCP servers employ, the need for MCP-specific taint semantics, and the fact that bugs often only materialize along complete tool-scoped execution paths. We present MCP-BiFlow, a bidirectional static analysis framework built around MCP-aware entrypoint recovery, protocol-specific taint modeling, and inter-procedural propagation analysis. Against a benchmark of 32 confirmed MCP vulnerability cases, MCP-BiFlow identifies 30 (93.8% recall), substantially outperforming CodeQL, Semgrep, Snyk Code, and MCPScan. Across 15,452 real-world MCP server repositories, MCP-BiFlow surfaces 549 overlap-compressed candidate clusters; manual review confirms 118 vulnerability paths in 87 servers, establishing unsafe propagation as a recurring failure mode that resists detection without protocol-aware recovery of both request-side and return-side flows.

1 Introduction

Large language model (LLM) applications are rapidly evolving from traditional chatbots into agents that can use tools and interact with external environments, e.g., for web retrieval, filesystem access, database querying, and code execution [14, 28, 55, 57, 59]. This shift is evident in agent frameworks, e.g., LangChain [25], AutoGen [58], and CrewAI [12], in coding agents, e.g., Claude Code [7] and OpenAI Codex [43], and in general-purpose assistants, e.g., Manus [30] and OpenClaw [47]. However, incompatible tool interfaces, schemas, and orchestration logic across platforms and frameworks hinder the portability and reuse of LLM agents [11, 23, 25, 41]. To address this issue, Anthropic introduced the Model Context Protocol (MCP) in late 2024 [5], a standardized host-client-server architecture in which servers expose tools, resources, and prompts to LLM applications through a uniform interface [23]. Within 16 months, the number of MCP servers has grown to nearly 60k [8]. Their utility is evidenced by strong adoption, with Microsoft’s Playwright MCP server exceeding 29k GitHub stars [32] and the community-maintained Figma MCP server reaching about 14k [19].

The growing adoption of MCP servers also has important security implications. In the MCP workflow, user input is received by an MCP host, where it is processed by an agent that decides whether to invoke a tool through an MCP client. The client then sends invocation arguments to the MCP server, which may access privileged local or remote resources and return results to the client, the host, or the end user. This workflow crosses multiple trust boundaries, as data flows from requester-controlled inputs and external content sources to privileged operations and downstream use of tool outputs. We therefore view MCP server security as a bidirectional trust-boundary data-flow problem. The first direction is **request-side propagation**, in which requester-controlled data is incorporated into tool parameters such as URLs, file paths, shell arguments, or query strings before reaching sensitive operations. Such flows can lead to vulnerabilities including SSRF, path traversal, command injection, and unsafe query execution. CVE-2025-53107 in `git-mcp-server`, for example, shows that git-related inputs can reach `child_process.exec` [39]. The second direction is **return-side propagation**, in which untrusted external content or sensitive internal data is incorporated into tool outputs and then consumed by the client, the host, or the model. CVE-2025-53355 in `mcp-server-kubernetes`, for example, shows that pod log content returned by the server can later influence a tool invocation [40]. In MCP settings, both directions are security-relevant because tool inputs may trigger privileged effects and tool outputs may re-enter LLM-mediated reasoning.

Existing work does not fully address this setting. Recent MCP security studies have mainly characterized ecosystem risks, attack patterns, and defenses [21, 23, 50, 66], while general-purpose analyzers such as CodeQL [15] and Semgrep [52] are not designed to recover MCP-specific interaction structure end to end. Systematic MCP analysis faces three concrete difficulties. (1) MCP servers expose externally reachable functionality through diverse tool registration, protocol-dispatch, and wrapper patterns rather than stable route abstractions. (2) Generic taint templates are often mismatched to MCP trust boundaries: on the request side, the relevant source is typically the decoded MCP argument payload, whereas on the return side the source may be externally fetched content and the sink is the protocol-visible return crossing the MCP boundary. (3) MCP vulnerabilities rarely manifest as short local source-to-sink pairs; they emerge through complete tool-scoped interaction paths spanning argument decoding, validation, helper forwarding, sensitive operations, and protocol-visible outputs.

We address these challenges with MCP-BiFlow, a bidirectional static analysis framework for MCP servers. MCP-BiFlow begins by recovering tool-specific entrypoints, linking publication sites with

*Haoyu Wang is the corresponding author (haoyuwang@hust.edu.cn).

the dispatch logic that routes a given invocation, so analysis is anchored to the handler reachable for each tool rather than a generic protocol wrapper. It then assigns taint semantics matched to MCP trust boundaries, distinguishing decoded request arguments, externally obtained values, sensitive operations, and protocol-visible returns. Finally, bidirectional interprocedural analysis over these recovered boundaries reconstructs full request-side and return-side propagation paths. The pipeline remains largely deterministic: LLM assistance is used only when local syntax is insufficient to determine whether a source remains materially attacker-controlled or whether a guard meaningfully blocks the flow. We evaluate MCP-BiFlow on both curated vulnerability cases and a broad corpus of open-source MCP servers. On a benchmark of 32 confirmed MCP vulnerability cases, MCP-BiFlow detects 30 (93.8% recall), substantially outperforming CodeQL, Semgrep, Snyk Code, and MCPScan. Across 15,452 real-world MCP server repositories, the tool produces 549 overlap-compressed candidate clusters spanning 424 servers; disclosure-oriented manual review confirms 118 vulnerability paths in 87 servers. In summary, we make the following contributions:

- **Propagation-based view of MCP vulnerabilities.** We frame MCP server security around two directions of unsafe propagation: *request-side propagation* and *return-side propagation*. This view captures how requester-controlled inputs, untrusted external content, and sensitive internal data cross trust boundaries to reach sensitive operations or model-consumable outputs.
- **Practical analysis framework.** We build MCP-BiFlow, which detects unsafe propagation in MCP servers by combining MCP-aware entrypoint recovery, protocol-specific taint modeling, and bidirectional interprocedural analysis. On a benchmark of 32 confirmed vulnerabilities, MCP-BiFlow reaches 93.8% recall.
- **Large-scale study of real-world MCP servers.** We apply MCP-BiFlow to 15,452 MCP servers across Python, JavaScript, and TypeScript projects. The pipeline surfaces 549 overlap-compressed candidate clusters across 424 servers, and final review confirms 118 vulnerability paths across 87 servers.

Artifact Availability. MCP-BiFlow and all artifacts are available at <https://doi.org/10.5281/zenodo.19335837>.

2 Background and Motivation

2.1 LLM Powered Autonomous Agents

LLM-powered autonomous agents have evolved from prompt-driven reasoning systems into software that can plan, retain state, and act on external environments [57, 62]. A typical architecture centers on an LLM augmented with planning, memory, and action modules, and among these components, *tool use* has emerged as the primary mechanism for capability expansion. By invoking APIs, querying databases, executing code, browsing the web, or interacting with desktop environments, agents can operate on real software and real-world resources rather than text alone [3, 26, 42, 51]. The scale of this shift is visible across the ecosystem. Frameworks such as LangChain and LangGraph have lowered the barrier to building long-running and multi-agent workflows [26, 27]; coding agents like Claude Code and Codex have brought tool use into real repositories and developer environments [7, 43, 45]; and general-purpose platforms such as OpenClaw have begun integrating chat interfaces, local execution, connected services, and reusable skills into unified

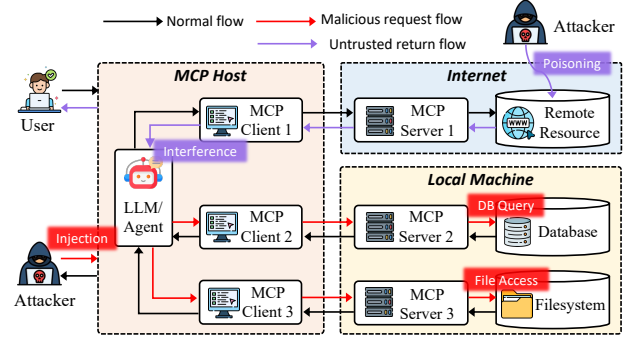


Figure 1: Data flows in MCP systems.

agent ecosystems [48, 49]. Alongside this growth, the mechanism of tool use has itself changed, moving from application-specific function calling and framework-level abstractions toward more reusable and interoperable interfaces, including protocolized connectors such as MCP and modular capability packages such as skills [23, 42, 44, 46].

2.2 The Rise of Model Context Protocol

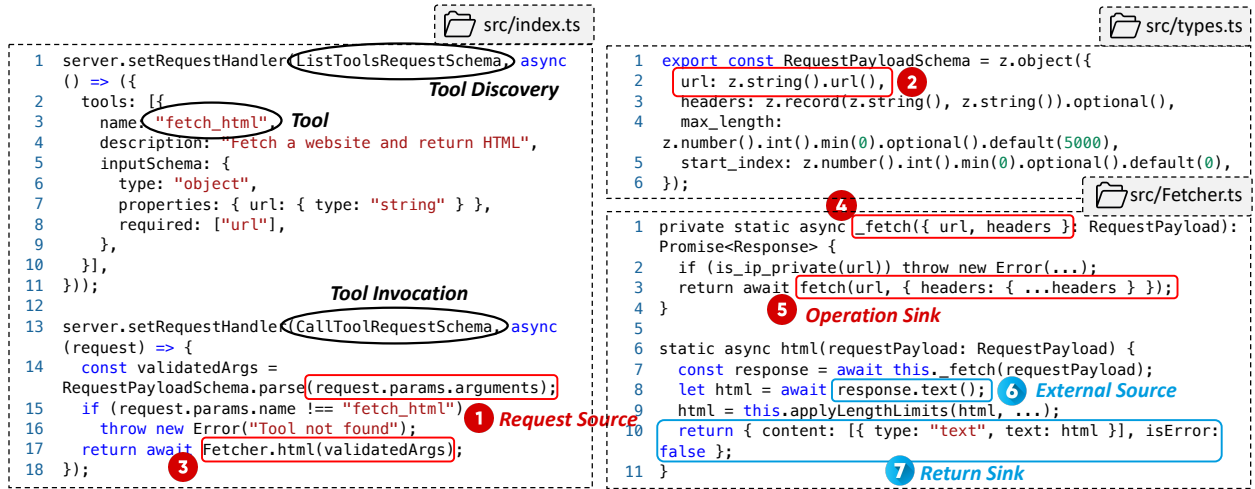
Model Context Protocol (MCP), introduced by Anthropic in late 2024, is an open standard for connecting AI applications to external tools, data sources, and workflows through a unified protocol interface [4, 35]. Compared with earlier function-calling APIs and framework-specific tool abstractions, MCP shifts tool integration toward protocol-level interoperability, including capability discovery, negotiation, and reuse across heterogeneous systems [4, 23, 33]. Since then, the ecosystem has expanded from mostly local integrations to standardized SDKs, broader platform adoption, and remote deployments [6, 31, 36]. At a high level, MCP involves three roles: an *MCP host*, an *MCP client*, and an *MCP server*. The host is the user-facing AI application, the client is the protocol component inside the host, and the server exposes tools, resources, or prompts [23, 33, 37]. In a typical workflow, as shown in Figure 1, the host decides whether an external capability is needed, the client discovers and invokes the corresponding server-side functionality, and the result is returned for further reasoning or final response generation [23, 33, 34, 37]. This architecture improves extensibility, but it also creates new failure modes: user-controlled inputs may reach privileged server-side operations, while untrusted external content or sensitive internal data may flow back through MCP outputs and affect downstream reasoning or user-visible responses [16–18, 38].

2.3 Problem Statement

Problem verview. In an MCP workflow, requester-controlled inputs may cross the host–client–server boundary and eventually influence privileged server-side operations, while externally obtained or sensitive server-side data may cross the MCP boundary through tool outputs and later affect the client, the host, the model, or the end user, as illustrated in Figure 1. We thus formulate MCP server security as a bidirectional trust-boundary data-flow problem. **Threat model.** We assume that the MCP host, client, and server are implemented by non-malicious parties and run in a non-compromised

Table 1: Main tool registration and dispatch patterns in MCP servers.

Pattern	Python Example	# Python	JS/TS Example	# JS/TS
Direct declaration	@<server_instance>.tool(name="x") def tool(...): ...	54	<server_instance>.tool("x", desc, schema, async (args) => { ... })	30
Explicit registration APIs	<server_instance>.add_tool(tool_fn, name="x") /* imperative registration */	1	<server_instance>.registerTool("x", {...}, async (args) => { ... })	14
Protocol-level dispatch	@<server_instance>.list_tools() @<server_instance>.call_tool() async def call_tool(...): ...	13	<server_instance>.setRequestHandler(ListToolsRequestSchema, ...) <server_instance>.setRequestHandler(CallToolRequestSchema, ...)	44
Other / mixed	/* custom abstraction, alternate implementation, or no matched tool-entry pattern */	32	/* custom wrapper, mixed styles, or no matched tool-entry pattern in quick pass */	12
Total		100		100


Figure 2: Running example based on CVE-2025-65513, marking request-side (red) and return-side (blue) propagation paths.

environment. Our focus is therefore not malicious infrastructure, but unsafe propagation caused by incomplete validation, insufficient sanitization, or unsafe handling of externally obtained or internally sensitive data. The attacker does not control server internals, source code, the host machine, or the deployment pipeline. Instead, the attacker acts through normal interaction channels, either by providing crafted inputs through the user-facing interface or by controlling external content sources consumed by MCP servers, such as third-party APIs, public repositories, web pages, or remote databases. Under this model, we consider two classes of unsafe propagation. **Request-side propagation** occurs when requester-controlled data reaches privileged or security-sensitive operations, including command execution, filesystem access, database queries, code evaluation, and outbound network requests. **Return-side propagation** occurs when untrusted external content or sensitive internal data is exposed through MCP-visible outputs, thereby crossing the server boundary and becoming available to the client, the host, or the end user. In MCP settings, such return-side flows are security-relevant because they may expose sensitive data or enable mixed chains in which returned content later affects downstream model reasoning or tool invocation. We exclude attacks that depend on a compromised code base, supply chain, or deployment

infrastructure, as well as issues of authentication, authorization, and business logic unless they manifest as unsafe cross-boundary data flows under this model.

2.4 Static Analysis Challenges in MCP Servers

Entrypoint discovery. Static analysis tools usually rely on framework models to identify externally reachable code, such as route decorators, handler registration APIs, and configuration driven bindings. Recovering these entrypoints matters because they mark where external input enters application logic and therefore seed reachability and taint analysis. This works well for frameworks such as Flask and FastAPI, where routes are explicit, but it often misses MCP servers, where externally reachable functionality is exposed through tool registration and dispatch. This limitation is beginning to receive attention in practice. Snyk Code, for example, has added support for recognizing MCP entrypoints [53], but current support mainly targets simple patterns such as FastMCP declarations. Real-world MCP servers use a much broader range of registration and dispatch styles. To characterize this diversity, we manually analyzed 100 Python and 100 JS/TS projects from the two largest MCP server ecosystems. Table 1 summarizes representative patterns. In the Python SDK, tools may be exposed through FastMCP decorators

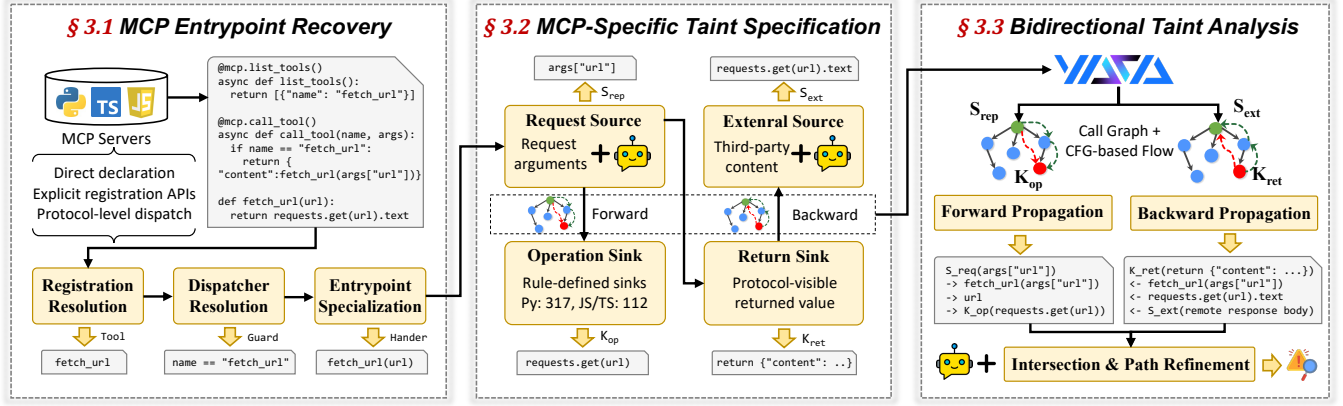


Figure 3: Overview of the MCP-BiFlow pipeline.

or lower level handlers such as `list_tools()` and `call_tool()`. In the TypeScript SDK, tools may be exposed either through direct registration APIs such as `tool(...)` or through protocol request handlers such as `setRequestHandler(CallToolRequestSchema, ...)`. Example in Figure 2 shows the latter pattern.

Challenge 1: MCP endpoints are exposed through diverse registration and dispatch patterns, making them difficult to recognize with existing static analysis models.

Taint semantics. Recovering tool endpoints alone is not enough to detect vulnerabilities in MCP, because generic repository-wide taint policies are too coarse for this setting. On the request side, many sensitive sinks remain relevant, but the corresponding sources are often hidden by MCP-specific protocol decoding, branch dispatch, schema parsing, and local aliasing. In Figure 2, the relevant source is not the entire request object, but the MCP argument payload in `request.params.arguments`. On the return side, the sink is defined by the recovered tool boundary, but the source often does not come from client input; it may instead be externally obtained content such as `response.text()` that is later exposed through the MCP response. Generic taint specifications do not model these request-boundary values and external content sources well.

Challenge 2: Generic source and sink templates do not capture the MCP-specific taint semantics needed to model request-side and return-side propagation.

Tool-scoped path recovery. MCP tool logic rarely appears as a short local source-to-sink pattern. On the request side, tool arguments are often parsed, validated, repackaged, and forwarded through helper layers before they reach a sensitive sink. On the return side, protocol-visible outputs are often assembled from external responses, local wrappers, and intermediate transformations before crossing the MCP boundary. In Figure 2, the red path (1, 3, 4, 5) captures request-side propagation from the MCP argument source `request.params.arguments` to the outbound `fetch` sink, while the blue path (6, 7) captures return-side propagation from the

external source `response.text()` to the protocol-visible return. The schema constraint at (2) further complicates the request-side path by refining the value without clearly removing its security relevance. In practice, MCP vulnerabilities are defined by complete tool-scoped interaction paths rather than isolated local patterns.

Challenge 3: MCP vulnerabilities often emerge only along complete tool-scoped interaction paths, so local source or sink reasoning alone is not enough.

3 Design of MCP-BiFlow

We design MCP-BiFlow as a static taint analysis framework for recovering flows that cross trust boundaries in real-world MCP servers. The overall workflow is shown in Figure 3. MCP-BiFlow integrates MCP Entrypoint Recovery (§ 3.1) to identify procedures reachable through tool interactions, MCP-Specific Taint Specification (§ 3.2) to model data derived from requests, externally obtained content, sensitive operations, and protocol-visible returns, and Bidirectional Interprocedural Taint Analysis (§ 3.3) to recover complete vulnerability paths from requests to sensitive operations and from external or internal sources to outputs.

3.1 MCP Entrypoint Recovery

To address Challenge 1, MCP-BiFlow performs MCP endpoint recovery before taint propagation. The goal is to recover logical tool endpoints even when tool publication, dispatch, and execution are implemented in different procedures. MCP-BiFlow first resolves publication patterns that expose tools to clients, then resolves dispatcher logic that maps tool names to concrete handlers, and finally combines the two into tool-specific analysis endpoints.

3.1.1 Registration Resolution. The first phase recovers publication evidence and converts it into publication facts $P = \{\langle t, n_p, h_p, \sigma_p \rangle\}$, where each fact records that tool name t is exposed at syntax node n_p with candidate handler h_p under witness σ_p . Our implementation models 10 publication variants across Python and JS/TS. Table 1 summarizes representative publication and dispatch patterns observed in our corpus, including direct declarations, explicit

registration APIs, and protocol-level publication patterns. For each matched site, MCP-BiFlow resolves aliases, imports, helper wrappers, and tool objects to a normalized tuple $\langle t, n_p, h_p, \sigma_p \rangle$. When a tool is represented as an object, the collector further resolves handler fields such as `execute`, `handler`, or `callback`, and schema fields such as `parameters`, `inputSchema`, or `schema`. In Figure 2, this phase produces a publication fact for `fetch_html` even though the concrete callee is revealed only later by the call handler.

3.1.2 Dispatcher Resolution. Publication evidence alone is insufficient because many MCP servers route invocations through generic protocol handlers or registries. MCP-BiFlow therefore extracts dispatch facts $D = \{\langle t, n_d, h_d, r, \sigma_d \rangle\}$ by resolving 16 dispatcher variants, where each fact records that invocations of tool t are handled by branch r rooted at node n_d and forwarded to handler h_d under witness σ_d . These idioms fall into four families. *Protocol dispatch* includes `setRequestHandler(CallToolRequestSchema, ...)` in JS/TS and `@call_tool()` handlers in Python. *Branch dispatch* covers equality tests, membership tests, `if/elif` chains, `switch`, and `match/case` over tool names, as well as outer JSON-RPC routers that first branch on `tools/call` and then on `params.name`. *Registry dispatch* covers literal maps, indexed assignments, `update(...)` extensions, helper-constructed registries, `map[name]`, `map.get(name)`, and getter helper indirection. *Reflective dispatch* covers patterns such as `getattr(self, f"handle_{name}")`. For each recovered dispatcher, MCP-BiFlow emits a fact $\langle t, n_d, h_d, r, \sigma_d \rangle$, where r records the matched branch scope and h_d is obtained by resolving forwarded helper calls, registry entries, object methods, or reflective targets. For the example in Figure 2, this phase isolates the branch guarded by `request.params.name == "fetch_html"` and resolves the forwarded callee to `Fetcher.html(validatedArgs)`.

3.1.3 Entrypoint Specialization. After recovering publication and dispatch facts, MCP-BiFlow specializes them into tool-specific analysis units. Let P_t and D_t denote the subsets of P and D associated with tool name t . For each tool t , MCP-BiFlow constructs recovered entrypoints of the form $e = \langle t, h, r \rangle$, where h is the resolved handler body and r is the relevant dispatcher scope. When both publication and dispatch facts are available, MCP-BiFlow prefers the handler recovered from dispatch to connect publication with concrete execution. When no dispatch fact is recovered for t , MCP-BiFlow falls back to the publication-side handler and sets $r = \top$ to denote an unbranched publication path. This specialization is important because multiple tools often share a protocol handler, helper layer, or registry object. By materializing $\langle t, h, r \rangle$, MCP-BiFlow seeds taint only along the path reachable for tool t , rather than across the entire dispatcher. Applied to Figure 2, the generic request handler is specialized into the concrete `fetch_html` entrypoint with handler `Fetcher.html` and the branch scope for the `fetch_html` case.

3.2 MCP-Specific Taint Specification

Once MCP entrypoints are recovered, MCP-BiFlow binds taint semantics to MCP-specific trust boundaries to address *Challenge 2*. A repository-wide source-sink policy would be too noisy: not every parameter is user-controlled, and not every return is exposed through the protocol. We therefore define four analysis object

classes as shown in Table 2. S_{req} and K_{op} support request-side misuse detection, while S_{ext} and K_{ret} support return-side reasoning about reflected or re-exposed untrusted content.

Table 2: Taint specification instantiated in Figure 2.

Category	Running Example Instance
Request boundary source (S_{req})	<code>request.params.arguments</code>
Sensitive operation sink (K_{op})	<code>fetch(url, { headers })</code>
External content source (S_{ext})	<code>response.text()</code>
Protocol return sink (K_{ret})	<code>return { content: [{ text: html }] }</code>

3.2.1 Request-Side Taint Semantics. For directly exposed MCP tools, formal parameters of the recovered handler are treated as request-boundary sources. For dispatcher-style handlers, taint is introduced only into the non-routing parameters or local variables of the matched branch, which prevents one tool path from polluting another. In addition, MCP-BiFlow lifts source semantics through common structured-input accessors, such as `arguments["path"]`, `arguments.get("url")`, object destructuring, and schema validated local aliases. This refinement is important because practical MCP handlers often unpack request objects immediately after validation and then propagate only the unpacked locals through deeper helper layers. To reduce over-approximation, MCP-BiFlow refines these candidates with a lightweight source-controllability check. Straightforward cases are handled deterministically, whereas semantically ambiguous candidates are resolved with LLM assistance only to determine whether the value remains materially requester-controlled after local validation or decoding. A simplified prompt template is shown in Figure 4. In Figure 2, the request-boundary source is `request.params.arguments`, even though the relevant `url` value is recovered only after specific parsing and validation. Sensitive operation sinks for request-side analysis are modeled through rule-driven specifications over concrete library calls. The current sink packs cover command and code execution, filesystem access, outbound network requests, database and query execution, and related side-effecting operations. Each sink specification binds taint checking to both a callee signature and the argument positions whose taint matters to security. In Figure 2, `fetch(url, { ... })` is treated as a sensitive operation sink because it issues an outbound request to a target derived from requester-controlled input, even though the code applies a private-IP guard.

3.2.2 Return-Side Taint Semantics. Return-side MCP vulnerabilities frequently originate not from client parameters, but from third-party content that the server fetches, reads, or derives from external state before returning it to the host or the model. MCP-BiFlow therefore models externally obtained values as a separate source class S_{ext} . Deterministic source recognizers cover common external-content carriers, including network response bodies, parsed remote JSON, file contents, subprocess `stdout/stderr`, git logs, markdown or documentation text, pod logs, and similar repository- or runtime-derived artifacts. However, purely syntactic evidence is not always sufficient: some wrappers transform, rename, or partially decode external content before returning it. For such cases, MCP-BiFlow slices the producer-side code around the recovered value and invokes an LLM only to classify whether the value is semantically an

Prompt

You are a software security analysis expert.

TASK

Decide whether the candidate value should be kept as a request-boundary source. A request-boundary source is a value that remains materially controlled by the MCP requester after local parsing or validation.

KEEP THE CANDIDATE IF

- it comes from MCP request arguments or request fields; and local checks do not reduce it to a fixed constant, closed enum, or other non-controllable value.

DROP THE CANDIDATE IF

- it is only routing metadata; it is replaced by a constant or fixed configuration value; or local validation reduces it to a closed safe domain.

INPUT FORMAT

```
{ "tool_name": "<Tool Name>", "candidate_expr": "<Candidate expression>",  
  "code_slice": "<Relevant handler / helper slice>" }
```

OUTPUT FORMAT

Return your result in the following JSON format:

```
{ "candidate_expr": "<Candidate expression>", "keep_as_source": true/false,  
  "decision": "keep" | "drop" | "uncertain", "reason": "<Short explanation>" }
```

Figure 4: Prompt template for request source identification.

untrusted external-content carrier. In Figure 2, `response.text()` is recognized as an external-content source because it reads the body of a remote HTTP response and directly contributes to the returned HTML. For return-side analysis, the return value of each recovered MCP handler is treated as a protocol-visible sink K_{ret} , because it crosses the MCP server boundary and becomes observable to the client, the host, or the model. Unlike a naïve “every return is a finding” policy, MCP-BiFlow reports a return sink only when the symbolic return value still depends on an externally obtained source or other tainted content. In the running example, the returned text: `html` field is a return sink because it exposes the remote response body to the MCP client.

3.3 Bidirectional Interprocedural Taint Analysis

After entrypoint recovery and taint specification, MCP-BiFlow performs bidirectional interprocedural taint analysis over the recovered handlers and helpers. It targets the two trust-boundary flows in our threat model: request-side propagation, in which request-boundary taint reaches a sensitive sink, and return-side propagation, in which externally obtained taint reaches a protocol-visible return. Let $CG = (F, C)$ denote the call graph over these procedures, where F is the set of procedures and $C \subseteq F \times F$ is the call relation. We track taint labels $\tau(v) \subseteq \{\text{req}, \text{ext}\}$, where `req` denotes request-boundary taint and `ext` denotes externally obtained taint. Figure 5 summarizes the core propagation rules.

3.3.1 Intra-Procedural Flow Construction. For each recovered procedure p , MCP-BiFlow builds intra-procedural control-flow graph $CFG_p = (N_p, E_p)$, where N_p is the set of statements in p and E_p is the set of control-flow edges. Over CFG_p , the analysis derives value-flow relations that capture how tainted data is propagated through assignments, computations, field accesses, container operations, calls, and returns. This step is important because MCP handlers often unpack structured request objects, pass them through helpers, and assemble return objects incrementally rather than flowing taint

Notations: $\alpha \in \{\text{req}, \text{ext}\}$: taint kind; $\tau(x) = \alpha$: value x carries taint α ; a_i : actual argument; p_i : formal parameter; ret_f : return value of function f ; r : assembled return object; K : sensitive sink.

Rules:

1. **Assignment Propagation:** $\tau(y) = \alpha \wedge \sigma : x := y \implies \tau(x) = \alpha$
2. **Field / Container Propagation:** $\tau(v) = \alpha \wedge \sigma : o.f := v \implies \tau(o.f) = \alpha$
 $\tau(o.f) = \alpha \wedge \sigma : x := o.f \implies \tau(x) = \alpha$
3. **Call-Argument Propagation:** $\tau(a_i) = \alpha \wedge \sigma : \text{call } f(a_1, \dots, a_n) \implies \tau(p_i) = \alpha$
4. **Return Propagation:** $\tau(ret_f) = \alpha \wedge \sigma : x := f(\dots) \implies \tau(x) = \alpha$
5. **Return-Assembly Propagation:** $\tau(v) = \alpha \wedge \sigma : r := \{\dots, v, \dots\} \implies \tau(r) = \alpha$
6. **Sink Reachability:** $\tau(v) = \alpha \wedge \sigma : K(v) \implies \text{emit candidate path}$

Figure 5: Core taint propagation rules in MCP-BiFlow.

directly from parameter to sink. At this level, taint continuity is governed by three recurring local patterns: if a tainted value y is assigned to x , then x remains tainted (Rule 1); if a tainted component v is written into a structured location such as `o.f`, or later read back from it, the taint is preserved across that object boundary (Rule 2); and if a response object r is assembled from a tainted component v , the enclosing return structure r is also tainted (Rule 5).

3.3.2 Inter-Procedural Forward Propagation. Forward propagation starts from S_{req} and follows interprocedural value dependencies toward K_{op} . Across call-graph edges, MCP-BiFlow propagates taint in two complementary steps summarized by Rules 3–4 of Figure 5: a tainted actual argument a_i taints the corresponding callee parameter p_i , and a tainted return value ret_f taints the receiving caller-side variable x . This process computes the request-reachable region

$$\mathcal{R}_{\text{op}}^f = \text{ReachForward}(S_{\text{req}}, CG),$$

which conservatively over-approximates all procedures and symbolic values that may be influenced by requester-controlled data. In Figure 2, taint originates at `request.params.arguments`, is preserved in `validatedArgs.url` after schema parsing, and then propagates through `Fetcher.html` and `_fetch` until it reaches the `fetch(url, ...)` operation sink. The private-IP check is treated as a guard on the path rather than as a transformation that removes request-boundary taint.

3.3.3 Inter-Procedural Backward Propagation. Backward propagation starts from K_{ret} and traces the symbolic contributors of each protocol-visible return value back toward candidate producers. The goal is to determine whether a returned value depends on an element of S_{ext} , rather than on a fixed literal, a local wrapper string, or a self-constructed status object. Operationally, this phase reuses the same summary relations in reverse: from a returned caller value x , Rule 4 steps back into the callee return ret_f , while Rule 5 peels apart an assembled response object r to recover the concrete producer values v that contribute to the protocol-visible field. Formally, this phase computes the return-reachable region

$$\mathcal{R}_{\text{ret}}^b = \text{ReachBackward}(K_{\text{ret}}, CG),$$

which includes assignments, helper returns, content-assembly statements, and producer-side I/O operations that contribute to the returned object. In the running example of Figure 2, the returned `html` field is traced backward through `applyLengthLimits(html, ...)`

to `response.text()`, which is already classified as an external-content source. This backward procedure is especially important for MCP because response-side risks are rarely visible as direct source-to-sink pairs; instead, the analysis must reconstruct how data is obtained, transformed, and eventually serialized into a client-visible return object.

3.3.4 Intersection and Path Refinement. The final phase validates complete taint paths by intersecting source-reachable and sink-reachable regions. Candidate findings are emitted only when the path construction induced by the earlier propagation steps still terminates at a sensitive sink, i.e., when some symbolic value v remains tainted at a sink invocation $K(v)$; this final reachability condition is captured by Rule 6 in Figure 5. For operation flows, MCP-BiFlow computes

$$\mathcal{I}_{\text{op}} = \mathcal{R}^f(S_{\text{req}}) \cap \mathcal{R}^b(K_{\text{op}}),$$

and retains a candidate only when the intersection forms a valid intra- or interprocedural data-flow path from a request-boundary source to a sensitive operation sink. For return flows, it computes

$$\mathcal{I}_{\text{ret}} = \mathcal{R}^f(S_{\text{ext}}) \cap \mathcal{R}^b(K_{\text{ret}}),$$

and retains a candidate only if the returned symbolic value still depends on an external-content source. After intersection, MCP-BiFlow performs path refinement to suppress candidates invalidated by effective guards or sanitizers, such as strict allowlists, canonicalization followed by root-membership checks for file paths, parameterized query APIs, or fixed-base URL constructions. The analysis records guard evidence, including schema checks, confinement checks, and quoting or normalization routines, along each candidate path. Straightforward cases are handled by deterministic suppression rules, whereas only semantically ambiguous guards are deferred to LLM-based adjudication to determine whether attacker influence is materially eliminated. Finally, equivalent paths that correspond to the same recovered MCP handler, sink site, and root cause are merged, and one representative finding is retained for downstream review.

4 Evaluation

Our evaluation focuses on three aspects of MCP-BiFlow: detection effectiveness on confirmed cases, applicability to real-world MCP servers, and the role of its major components.

RQ1: How well does MCP-BiFlow detect confirmed MCP vulnerabilities relative to existing tools? We study this question on a benchmark of confirmed MCP vulnerability cases, using case-level detections and recall for comparison.

RQ2: What vulnerabilities can MCP-BiFlow uncover in real-world MCP servers? To answer this question, we apply MCP-BiFlow to real-world open-source MCP server repositories and manually review the resulting candidate findings.

RQ3: Which components are responsible for MCP-BiFlow's coverage and triage behavior? An ablation study is used to isolate how the major MCP-aware components affect reviewed-instance coverage and triage outcomes.

4.1 Evaluation Setup

Tool implementation. We implemented a prototype of MCP-BiFlow on top of the open-source static analysis framework YASA [20]. The pipeline consists of three main stages: MCP entrypoint recovery, MCP-specific taint specification, and bidirectional interprocedural taint analysis. Semantically ambiguous source and guard cases are handled through the LLM-assisted adjudication.

Environment. All experiments were conducted on a server running Ubuntu 22.04, equipped with two 64-core AMD EPYC 9954 processors, 1024 GB of RAM, and six NVIDIA A100 GPUs, each with 80 GB of memory. The LLM-assisted analysis in our pipeline was powered by the gpt-5.3-codex-medium API.

Baselines. We compare MCP-BiFlow against four baseline tools: CodeQL [15], Semgrep [52], Snyk Code [54], and MCPScan [2]. CodeQL serves as a representative query-based static analysis baseline, while Semgrep represents a lightweight rule-based analyzer. Snyk Code is a commercial semantic SAST baseline with announced support for MCP-specific input sources, mainly in FastMCP-based implementations. MCPScan is the closest MCP-oriented baseline in our comparison, combining rule matching with LLM-assisted triage.

Dataset. For RQ1, we collected MCP-related vulnerability cases from public sources including GitHub Security Advisories, the GitLab Advisory Database, NVD, OSV, and selected security reports. Cases were retained through manual curation requiring clear MCP relevance, convincing security evidence, and implementations in Python, JavaScript, or TypeScript. The finalized comparative benchmark contains 32 confirmed vulnerability cases spanning several categories, with both request-side and return-side relevant scenarios represented. For RQ2, we gathered MCP server repositories from three public registries: *mcp.so*, *PulseMCP*, and *MCPWorld*. Starting from 75,380 GitHub repository links, URL normalization and cross-source deduplication produced 63,639 unique repositories. We then enriched each repository with GitHub metadata and applied rule-based filtering to retain accessible repositories likely to implement MCP servers, removing clients, SDKs, examples, templates, registries, and archived or forked projects. Our analysis concentrates on Python, JavaScript, and TypeScript because these three languages collectively account for nearly 80% of the collected MCP servers, making them the most representative target for large-scale study. The corpus contains 15,452 repositories: 8,047 in Python (52.1%), 4,977 in TypeScript (32.2%), and 2,428 in JavaScript (15.7%).

4.2 RQ1: Detection Effectiveness

To answer RQ1, we evaluate MCP-BiFlow on the MCP vulnerability benchmark of 32 confirmed cases drawn from Python, JavaScript, and TypeScript MCP servers, covering command and code execution, filesystem access, database and query execution, and SSRF. Because the benchmark contains only vulnerable cases, we report case-level detections and recall rather than precision. As shown in Table 3, MCP-BiFlow detects **30 of 32 cases** (93.8% recall).

The benchmark is dominated by request-side cases and mixed chains rather than pure return-only disclosures, since public MCP advisories are typically anchored in a concrete side-effecting sink. Rows marked in both the *Req.* and *Resp.* columns should be read as *mixed chains*: attacker-relevant content crosses the MCP return boundary and later contributes to a downstream request-side effect.

Table 3: Case-level comparison on the 32 confirmed MCP vulnerability cases.

Type	CVE ID	Server Name	Lang.	Req.	Resp.	CodeQL	Semgrep	Snyk Code	MCPScan	MCP-BiFlow
Command / Code Execution	CVE-2025-11202	win-cli-mcp-server	JS/TS	✓		○	●	○	●	●
	CVE-2025-52573	ios-simulator-mcp	JS/TS	✓		○	○	●	●	●
	CVE-2025-5277	aws-mcp-server	Py	✓		○	○	○	●	●
	CVE-2025-53818	github-kanban-mcp-server	JS/TS	✓		○	○	○	○	●
	CVE-2025-53832	lara-mcp	JS/TS	✓		○	○	●	○	●
	CVE-2025-59834	adb-mcp	JS/TS	✓		○	○	○	○	●
	CVE-2025-63603	mcp-server-data-exploration	Py	✓		○	●	○	●	●
	CVE-2025-69256	serverless	JS/TS	✓		○	○	○	○	●
	CVE-2026-26029	sf-mcp-server	JS/TS	✓		○	○	○	●	●
	CVE-2025-53107	git-mcp-server	JS/TS	✓	✓	○	○	○	○	●
	CVE-2025-53355	mcp-server-kubernetes	JS/TS	✓	✓	○	○	○	●	●
	CVE-2025-66404	mcp-server-kubernetes	JS/TS	✓	✓	○	○	○	●	●
	CVE-2025-58358	markdownify-mcp	JS/TS	✓	✓	●	○	●	●	●
	CVE-2025-54073	mcp-package-docs	JS/TS	✓	✓	○	○	●	○	●
	CVE-2025-53967	Figma-Context-MCP	JS/TS	✓	✓	○	○	○	○	●
	CVE-2025-53372	node-code-sandbox-mcp	JS/TS	✓	✓	○	○	○	●	●
	CVE-2025-59376	mcp-kubernetes-server	Py	✓		○	○	○	●	●
	CVE-2025-59377	mcp-kubernetes-server	Py	✓		○	●	○	●	●
	CVE-2026-2178	xcode-mcp-server	JS/TS	✓		○	○	○	○	○
Filesystem Access	CVE-2025-53109	server-file-system	JS/TS	✓		○	●	●	○	●
	CVE-2025-53110	server-file-system	JS/TS	✓		○	●	●	○	●
	CVE-2025-67364	fast-file-system-mcp	JS/TS	✓		○	●	●	○	●
	CVE-2025-68143	mcp-server-git	Py	✓		○	○	○	-	●
	CVE-2025-68144	mcp-server-git	Py	✓		○	○	○	-	●
	CVE-2025-68145	mcp-server-git	Py	✓		○	○	○	-	●
	CVE-2026-27735	mcp-server-git	Py	✓		○	○	○	-	●
	CVE-2026-27825	mcp-atlassian	Py	✓		○	○	○	-	●
	CVE-2025-61685	mastra	JS/TS	✓		○	○	○	○	○
Database / Query Execution	CVE-2025-59333	mcp-database-server	JS/TS	✓		○	●	○	-	●
	CVE-2025-5276	markdownify-mcp	JS/TS	✓		○	○	●	○	●
SSRF	CVE-2025-65513	fetch-mcp	JS/TS	✓	✓	○	○	●	○	●
Total Detected (32 Cases)						1	8	10	11	30

Note. Lang.: Py = Python and JS/TS = JavaScript/TypeScript. In the **Req.** and **Resp.** columns, ✓ indicates that the case is included in the corresponding request-side or return-side split; rows marked in both columns denote mixed chains. In the tool columns, ● denotes a reviewed case-compatible detection and ○ denotes no credited detection. A dash ("-") indicates that the case lies outside the evaluated capability boundary of that tool.

Against this benchmark, CodeQL detects only 1 case, as its default queries do not model MCP-specific tool exposure, handler parameters, or protocol-visible returns as analysis boundaries. Semgrep reaches 8 cases when the bug manifests as a local API misuse, but loses coverage once the flow spans helpers, wrappers, or indirect dispatch. Snyk Code detects 10 cases, partly because it already recognizes MCP server sources and endpoints in FastMCP and the official Python and TypeScript SDKs, yet it still misses vulnerabilities that require broader dispatch recovery, wrapper traversal, or return-side propagation. MCPScan is the strongest baseline at 11 cases, though it falls well short of MCP-BiFlow, showing that MCP-specific rules are insufficient without end-to-end recovery of both propagation directions.

The two false negatives mark the current implementation boundary in JavaScript. CVE-2025-65513 requires rebinding decoded MCP arguments into object-destructured parameters with default values, while CVE-2025-61685 requires propagating taint into callback parameters across a higher-order iteration and closure boundary.

Both misses trace back to current gaps in parameter rebinding and cross-closure taint propagation.

4.3 RQ2: Real-World Applicability

To answer RQ2, we ran MCP-BiFlow on 15,452 MCP server repositories and manually reviewed the highest-risk outputs. Near-duplicate traces from the same server and nearby sink site were merged into *overlap-compressed candidate clusters*, yielding 549 clusters across 424 servers. Disclosure review then consolidated surviving evidence into distinct *vulnerability paths*, confirming 118 paths across 87 servers. In the first review pass, 278 clusters were marked reportable, 55 required additional manual judgment, and 216 were rejected (details are included in our artifact). Rejected clusters fell into a few recurring patterns: user input did not actually control the request target in 122 cases, the target was effectively fixed or configuration-only in 53, and local validation made shell interpolation non-exploitable in 21. The final disclosure review applied stricter criteria, confirming 118 paths and rejecting the remaining 431 cases. The gap between candidates and confirmed paths reflects

Table 4: Primary sink functions in the confirmed real-world MCP vulnerabilities ($N = 118$).

Vulnerability Type	Sink Function / Primitive	# Paths
Code Execution	execAsync	60
	execSync	28
	exec	9
	eval	5
	Function constructor	3
	conn.exec	2
	asyncio.create_subprocess_shell	2
Query Execution	subprocess.run	2
	connection.query	3
	client.query	1
SSRF	db.unsafe	1
	fetch	2

recall-oriented candidate generation paired with conservative disclosure standards rather than a direct precision estimate. Among the 118 confirmed paths, 111 involve code execution, 5 involve query execution, and 2 involve SSRF. The primary sink functions are summarized in Table 4.

We next present two representative case studies corresponding to the two propagation directions in our threat model: a request-side path from MCP tool input to a privileged execution sink, and a return-side path in which externally obtained content crosses trust boundaries and contributes to unsafe downstream behavior.

Case study #1: Request-side propagation. Our first case comes from `bhouston/mcp-server-text-editor`, an MCP implementation of Claude’s built-in text editor. Although the tool is presented as a file-viewing and editing interface, the `view` path invokes `execSync` when the supplied path refers to a directory. As shown in Figure 6, the path parameter is requester-controlled and verified only to be absolute before being interpolated into a shell command. That check offers no protection against shell injection, since an absolute path can still carry shell-significant characters. The resulting flow is a representative request-side propagation path: MCP-visible input crosses the server boundary and reaches a privileged execution sink, allowing a seemingly benign tool to be steered into arbitrary shell execution on the host.

Case study #2: Externally sourced propagation and return-side risk. `LyuboslavLyubenov/search-solodit-mcp` is our second case, a server that searches and retrieves public Solodit vulnerability reports. Here, the security-relevant data does not originate from a local tool argument flowing directly into a sink. Instead, the `get-by-slug` tool accepts a user-controlled slug, fetches report data from the Solodit API, extracts `json[0].result.data`, and passes it directly to `eval`, as shown in Figure 7, with no structured decoding or validation applied before evaluation. This case illustrates that unsafe propagation in MCP servers is not limited to direct request-side misuse: externally obtained content can enter through a retrieval path, remain security-relevant across the MCP interaction, and reach a dangerous operation several steps later. In our threat model, such flows connect closely to return-side risk, because the same externally sourced content is delivered through an MCP-visible retrieval interface and may participate in mixed chains that influence later model reasoning or tool invocations.

```

1 export const toolParameters = {
2   command: z.enum(["view", "create", "str_replace", "insert",
3     "undo_edit"]),
4   path: z.string().describe("Absolute path to file or
5     directory"),
6   ...
7 };
8
9 const textEditorInternal = async ({ command, path: filePath }:
10   Parameters) => {
11   if (!path.isAbsolute(filePath)) {
12     throw new Error("Path must be absolute");
13   }
14
15   switch (command) {
16     case "view": {
17       const stats = await fs.stat(filePath).catch(() => null);
18       if (stats.isDirectory()) {
19         const output = execSync(
20           `find "${filePath}" -type f -not -path "*/\\.*" -
21             maxdepth 2 | sort`,
22           { encoding: "utf8" },
23         );
24         return { success: true, content: output };
25       }
26     }
27   }
28 }

```

Figure 6: Request-side propagation to command execution in `mcp-server-text-editor`.

```

1 server.registerTool(
2   "get-by-slug",
3   {
4     description: "Gets a solodit vulnerability report based on
5       a slug",
6     inputSchema: { slug: zod.string().nonempty() },
7   },
8   async (args) => {
9     const slug = args.slug.replace(/^"|$/g, "");
10    const findings = await getFinding(slug);
11    return {
12      content: [{ type: "text", text: findings?.content ??
13        "Report not found" }],
14    };
15  });
16
17 export default async function getFinding(slug: string) {
18   const response = await fetch(buildSoloditUrl(slug));
19   const json = await response.json();
20   const contentResult = json[0].result.data;
21   const obj = eval(`${contentResult}`);
22   return obj;
23 }

```

Figure 7: Externally sourced propagation with return-side risk in `search-solodit-mcp`.

4.4 RQ3: Ablation Study

We answer RQ3 through a variant-based ablation on the reviewed real-world instance set. Table 5 compares the full pipeline against three ablated variants, each omitting one core component: M1 for MCP endpoint recovery, M2 for LLM-assisted source identification, and M3 for LLM-assisted guard and sanitizer validation. Results are reported over reviewed instances rather than unique servers. The reviewed set contains 859 instances in total, of which 424 are confirmed, 302 unresolved, and 133 rejected. These counts are not directly comparable to the server- and cluster-level figures in RQ2, since a single server may contribute multiple reviewed instances with different paths, sinks, or review outcomes. M1 and M2 both bear primarily on coverage. Removing M1 reduces surfaced instances from 859 to 636 and confirmed instances from 424

Table 5: Ablation on reviewed real-world instances.

Variant	Confirmed	Unresolved	Rejected	Retained	Total
M1+M2+M3	424	302	133	726	859
w/o M1	287	251	98	538	636
w/o M2	324	210	87	534	621
w/o M3	424	368	67	792	859

Note. The reviewed set contains 859 reviewed instances: 424 confirmed, 302 unresolved, and 133 rejected. These are reviewed instances rather than unique servers; one server may contribute multiple instances. M1: MCP endpoint recovery. M2: LLM-assisted source identification. M3: LLM-assisted guard/sanitizer validation. **Retained** = Confirmed + Unresolved.

to 287, which reflects how much MCP-specific endpoint recovery contributes to exposing handlers hidden behind protocol dispatch and wrapper code. Removing M2 leaves 621 surfaced instances and 324 confirmed ones, indicating that many findings depend on recovering source semantics beyond direct tool parameters. M3 plays a distinct role. Its removal leaves the confirmed count unchanged at 424, but unresolved instances rise from 302 to 368 and rejected instances fall from 133 to 67. The component therefore improves triage quality rather than raw coverage: it converts a substantial fraction of otherwise ambiguous cases into defensible reject decisions, bringing the retained set down from 792 to 726. Taken together, the ablation indicates that M1 and M2 govern finding coverage while M3 governs the reliability of triage outcomes.

5 Discussion

Mitigation. Our findings suggest that securing MCP servers requires attention to both propagation directions. On the request side, requester-controlled values should be prevented from reaching execution boundaries through shell commands, unconstrained paths, URLs, or query strings. On the return side, externally obtained or sensitive internal data should not be treated as benign tool output by default, since it may later re-enter prompts or downstream tool invocations. Structured argument validation, explicit allowlisting, and deployment safeguards such as sandboxing, filesystem restrictions, and network egress control together reduce the attack surface that MCP tool interfaces expose.

Generality. The core design of MCP-BiFlow is not tied to a specific MCP SDK or language. It is organized around three abstractions: endpoint recovery, trust-boundary-specific source and sink modeling, and bidirectional interprocedural propagation. The current implementation targets Python, JavaScript, and TypeScript MCP servers, but the same formulation can accommodate other languages by supplying language-specific support for registration, dispatch, and sink patterns. More broadly, the approach is applicable to other tool-serving frameworks that connect LLM-powered applications to privileged capabilities.

Limitations. MCP-BiFlow targets unsafe cross-boundary data flows rather than the full space of MCP security problems. Authentication, authorization, business-logic, and deployment flaws fall outside its scope unless they manifest as analyzable data-flow violations, and metadata-level attacks such as tool-description poisoning are not directly addressed unless those artifacts propagate into executable code paths captured by the analysis. On the practical side, false positives remain the main challenge: the pipeline may over-approximate attacker control, overlook effective local constraints,

or flag flows whose targets are fixed, configuration-only, or otherwise not meaningfully attacker-influenced. Completeness can also suffer when servers rely on reflective dispatch, runtime-generated handlers, or framework-specific wrappers. Finally, the real-world evaluation is disclosure-oriented, and semantically ambiguous cases require LLM-assisted adjudication.

6 Related Work

Research on MCP security. As MCP becomes a foundational interface for tool-augmented AI ecosystems, its security and reliability have drawn increasing attention [21, 23]. Most prior work characterizes the MCP attack surface rather than statically recovering vulnerable data flows in server implementations. Hasan et al. [21] measure open-source MCP servers and identify common vulnerability categories and maintenance weaknesses, while Hou et al. [23] survey the broader MCP landscape, threat model, and open research directions. Zhao et al. [66] present a taxonomy of malicious MCP server behaviors and demonstrate proof-of-concept exploits, and Radosevich and Halloran [50] show that current MCP workflows remain vulnerable to severe code-execution and privilege-escalation attacks. Other work examines preference manipulation in open MCP marketplaces [56], parasitic toolchain attacks across MCP components [65], and benchmark-driven security evaluation through MCPsecBench [61]. Complementary defenses explore OAuth-enhanced tool definitions, protocol hardening, secure gateways, and layered detection mechanisms [9, 10, 24, 60]. We take a different angle, using static analysis to recover request-side and return-side trust-boundary violations directly in MCP server code.

Security of tool-using LLM agents. Beyond MCP, a growing body of work studies the security risks inherent in tool-using LLM systems and autonomous agents [13, 22, 28, 55, 63]. Tool use expands the attack surface of LLM applications by coupling model reasoning with external actions and untrusted content channels [13, 22, 64]. Indirect prompt injection is a particularly well-studied instance of this risk: attacker-controlled content retrieved from external sources can re-enter the model’s context and shape subsequent reasoning or tool invocations [1, 29]. This literature connects closely to our return-side threat model, yet it typically analyzes system-level attacks, benchmarks, host-side failures, or agent-level taxonomies without performing static reasoning over MCP server code [63, 64]. We make these risks analyzable at the server level by treating protocol-visible returns as first-class security boundaries and tracing the paths through which external content reaches them.

7 Conclusion

As the MCP take on a growing role in connecting LLM agents to external tools, the trust-boundary crossings they introduce create classes of unsafe data flow that conventional analyzers are poorly equipped to detect. We framed this as a bidirectional problem and built MCP-BiFlow around MCP endpoint recovery, protocol-specific taint modeling, and interprocedural analysis, covering both request-side and return-side propagation within a unified framework. On a benchmark of 32 confirmed vulnerability cases, MCP-BiFlow detects 30, outperforming CodeQL, Semgrep, Snyk Code, and MCPScan. Across 15,452 real-world repositories, it uncovers 118 vulnerability paths in 87 servers, confirming that insecure data

flow is not an edge case in the MCP ecosystem but a recurring structural weakness.

References

- [1] Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec 2023, Copenhagen, Denmark, 30 November 2023*, Maura Pintor, Xinyun Chen, and Florian Tramèr (Eds.). ACM, 79–90. doi:10.1145/3605764.3623985
- [2] Ant Group. 2026. MCPScan. <https://github.com/antgroup/MCPScan>. Accessed 2026-03-25.
- [3] Anthropic. 2024. Developing a Computer Use Model. <https://www.anthropic.com/news/developing-computer-use>. Accessed 2026-03-25.
- [4] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed 2026-03-25.
- [5] Anthropic. 2024. Model Context Protocol Documentation. <https://modelcontextprotocol.io/>. Accessed: 2026-03-26.
- [6] Anthropic. 2025. Code Execution with MCP: Building More Efficient AI Agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>. Accessed 2026-03-25.
- [7] Anthropic. 2026. Claude Code Overview. <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>. Accessed 2026-03-25.
- [8] Baidu. 2025. MCP World: Explore a Vast Collection of MCP Servers. <https://www.mcpworld.com/>. Accessed: 2026-03-26.
- [9] Manish Bhatt, Vineeth Sai Narajala, and Idan Habler. 2025. ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP) by using OAuth-Enhanced Tool Definitions and Policy-Based Access Control. *CoRR* abs/2506.01333 (2025). arXiv:2506.01333 doi:10.48550/ARXIV.2506.01333
- [10] Ivo Brett. 2025. Simplified and Secure MCP Gateways for Enterprise AI Integration. *CoRR* abs/2504.19997 (2025). arXiv:2504.19997 doi:10.48550/ARXIV.2504.19997
- [11] ByteDance. 2024. Coze plugin store. <https://www.coze.com/store/plugin>.
- [12] CrewAI. 2025. CrewAI. <https://github.com/crewAIInc/crewAI>. Accessed: 2026-03-26.
- [13] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlu Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. 2024. AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. *CoRR* abs/2406.02630 (2024). arXiv:2406.02630 doi:10.48550/ARXIV.2406.02630
- [14] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, Yunjun Gao, Yingcai Wu, and Shouling Ji. 2024. Navigating the Risks: A Survey of Security, Privacy, and Ethics Threats in LLM-Based Agents. *CoRR* abs/2411.09523 (2024). arXiv:2411.09523 doi:10.48550/ARXIV.2411.09523
- [15] GitHub. 2026. CodeQL. <https://codeql.github.com/>. Accessed 2026-03-25.
- [16] GitHub Security Advisory. 2025. GHSA-3q26-f695-pp76: @cyanheads/git-mcp-server Command Injection / RCE. <https://github.com/advisories/GHSA-3q26-f695-pp76>. CVE-2025-53107. Accessed: 2026-03-25.
- [17] GitHub Security Advisory. 2025. GHSA-gjv4-ghm7-q58q: mcp-server-kubernetes Command Injection / RCE. <https://github.com/advisories/GHSA-gjv4-ghm7-q58q>. CVE-2025-53355. Accessed: 2026-03-25.
- [18] GitHub Security Advisory. 2025. GHSA-hc55-p739-j48w: @modelcontextprotocol/server-file-system Path Traversal / Unauthorized File Access. <https://github.com/modelcontextprotocol/servers-security/advisories/GHSA-hc55-p739-j48w>. CVE-2025-53110. Accessed: 2026-03-25.
- [19] GLiPS. 2025. GLiPS/Figma-Context-MCP. <https://github.com/GLiPS/Figma-Context-MCP>. Accessed: 2026-03-26.
- [20] Ant Group. 2025. YASA (Yet Another Static Analyzer). <https://github.com/antgroup/YASA-Engine>. Accessed: 2026-03-26.
- [21] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2025. Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. *CoRR* abs/2506.13538 (2025). arXiv:2506.13538 doi:10.48550/ARXIV.2506.13538
- [22] Yifeng He, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. 2024. Security of AI Agents. *CoRR* abs/2406.08689 (2024). arXiv:2406.08689 doi:10.48550/ARXIV.2406.08689
- [23] Xinyi Hou, Yanjie Zhao, Sheng Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *CoRR* abs/2503.23278 (2025). arXiv:2503.23278 doi:10.48550/ARXIV.2503.23278
- [24] Sonu Kumar, Anubhav Girdhar, Ritesh Patil, and Divyansh Tripathi. 2025. MCP Guardian: A Security-First Layer for Safeguarding MCP-Based AI System. *CoRR* abs/2504.12757 (2025). arXiv:2504.12757 doi:10.48550/ARXIV.2504.12757
- [25] LangChain. 2022. LangChain: Framework for developing applications powered by language models. <https://github.com/langchain-ai/langchain>.
- [26] LangChain. 2024. Announcing LangGraph v0.1 & LangGraph Cloud: Running Agents at Scale, Reliably. <https://blog.langchain.dev/langgraph-cloud>. Accessed 2026-03-25.
- [27] LangChain. 2024. LangChain v0.1.0. <https://blog.langchain.dev/langchain-v0-1-0/>. Accessed 2026-03-25.
- [28] Yuanjun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. *CoRR* abs/2401.05459 (2024). arXiv:2401.05459 doi:10.48550/ARXIV.2401.05459
- [29] Tong Liu, Zizhuang Deng, Guozhu Meng, Yuekang Li, and Kai Chen. 2023. Demystifying RCE Vulnerabilities in LLM-Integrated Apps. *CoRR* abs/2309.02926 (2023). arXiv:2309.02926 doi:10.48550/ARXIV.2309.02926
- [30] Manus. 2025. Manus. <https://manus.im/>. Accessed: 2026-03-26.
- [31] MCP Core Maintainers. 2025. Exploring the Future of MCP Transports. <https://blog.modelcontextprotocol.io/posts/2025-12-19-mcp-transport-future/>. Accessed 2026-03-25.
- [32] Microsoft. 2025. microsoft/playwright-mcp. <https://github.com/microsoft/playwright-mcp>. Accessed: 2026-03-26.
- [33] Model Context Protocol. 2025. Architecture Overview. <https://modelcontextprotocol.io/docs/learn/architecture>. Accessed 2026-03-25.
- [34] Model Context Protocol. 2025. Lifecycle. <https://modelcontextprotocol.io/specification/2025-03-26/basic/lifecycle>. Accessed 2026-03-25.
- [35] Model Context Protocol. 2025. Model Context Protocol: Introduction. <https://modelcontextprotocol.io/docs/getting-started/intro>. Accessed 2026-03-25.
- [36] Model Context Protocol. 2025. Model Context Protocol Specification. <https://modelcontextprotocol.io/specification/2025-11-25>. Accessed 2026-03-25.
- [37] Model Context Protocol. 2025. Understanding MCP Clients. <https://modelcontextprotocol.io/docs/learn/client-concepts>. Accessed 2026-03-25.
- [38] National Vulnerability Database. 2025. CVE-2025-5277: aws-mcp-server Command Injection Vulnerability. <https://nvd.nist.gov/vuln/detail/CVE-2025-5277>. Accessed: 2026-03-25.
- [39] National Vulnerability Database. 2025. CVE-2025-53107. <https://nvd.nist.gov/vuln/detail/CVE-2025-53107>. Accessed: 2026-03-26.
- [40] National Vulnerability Database. 2025. CVE-2025-53355. <https://nvd.nist.gov/vuln/detail/CVE-2025-53355>. Accessed: 2026-03-26.
- [41] OpenAI. 2023. ChatGPT plugins. <https://openai.com/index/chatgpt-plugins/>.
- [42] OpenAI. 2025. Function Calling. <https://developers.openai.com/api/docs/guides/function-calling/>. Accessed 2026-03-25.
- [43] OpenAI. 2025. Introducing Codex. <https://openai.com/index/introducing-codex/>. Accessed 2026-03-25.
- [44] OpenAI. 2026. Agent Skills – Codex. <https://developers.openai.com/codex/skills/>. Accessed 2026-03-25.
- [45] OpenAI. 2026. Introducing the Codex App. <https://openai.com/index/introducing-the-codex-app/>. Accessed 2026-03-25.
- [46] OpenAI. 2026. Skills. <https://developers.openai.com/api/docs/guides/tools-skills/>. Accessed 2026-03-25.
- [47] OpenClaw. 2025. OpenClaw. <https://github.com/cyberagiinc/OpenClaw>. Accessed: 2026-03-26.
- [48] OpenClaw. 2026. Introducing OpenClaw. <https://openclaw.ai/blog/introducing-openclaw>. Accessed 2026-03-25.
- [49] OpenClaw. 2026. Trust – OpenClaw. <https://trust.openclaw.ai/>. Accessed 2026-03-25.
- [50] Brandon Radosevich and John Halloran. 2025. MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits. *CoRR* abs/2504.03767 (2025). arXiv:2504.03767 doi:10.48550/ARXIV.2504.03767
- [51] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *CoRR* abs/2302.04761 (2023). arXiv:2302.04761 doi:10.48550/ARXIV.2302.04761
- [52] Semgrep. 2026. Semgrep. <https://github.com/semgrep/semgrep>. Accessed 2026-03-25.
- [53] Snyk. 2025. Snyk Code: Support for MCP Server Sources. <https://updates.snyk.io/snyk-code-support-for-mcp-server-sources-319181/>. Accessed: 2026-03-26.
- [54] Snyk. 2026. Snyk Code. <https://snyk.io/product/snyk-code/>. Accessed 2026-03-25.
- [55] Yuntao Wang, Yanghe Pan, Zhou Su, Yi Deng, Quan Zhao, Linkang Du, Tom H. Luan, Jiawen Kang, and Dusit Niyato. 2025. Large Model Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends. arXiv:2409.14457 [cs.AI] <https://arxiv.org/abs/2409.14457>
- [56] Zihan Wang, Hongwei Li, Rui Zhang, Yu Liu, Wenbo Jiang, Wenshu Fan, Qingchuan Zhao, and Guowen Xu. 2025. MPMA: Preference Manipulation Attack Against Model Context Protocol. *CoRR* abs/2505.11154 (2025). arXiv:2505.11154 doi:10.48550/ARXIV.2505.11154
- [57] Lilian Weng. 2023. LLM-powered Autonomous Agents. *Lil'Log* (June 2023). <https://lilianweng.github.io/posts/2023-06-23-agent/>
- [58] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*

- (2023). arXiv:2308.08155 [cs.AI] <https://arxiv.org/abs/2308.08155>
- [59] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv:2309.07864 [cs.AI] <https://arxiv.org/abs/2309.07864>
- [60] Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. 2025. MCP-Guard: A Defense Framework for Model Context Protocol Integrity in Large Language Model Applications. CoRR abs/2508.10991 (2025). arXiv:2508.10991 doi:10.48550/ARXIV.2508.10991
- [61] Yixuan Yang, Daoyuan Wu, and Yufan Chen. 2025. MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols. CoRR abs/2508.13220 (2025). arXiv:2508.13220 doi:10.48550/ARXIV.2508.13220
- [62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. CoRR abs/2210.03629 (2022). arXiv:2210.03629 doi:10.48550/ARXIV.2210.03629
- [63] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pang, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. 2025. A Survey on Trustworthy LLM Agents: Threats and Countermeasures. arXiv preprint arXiv:2503.09648 (2025).
- [64] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2024. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. CoRR abs/2410.02644 (2024). arXiv:2410.02644 doi:10.48550/ARXIV.2410.02644
- [65] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2025. Mind Your Server: A Systematic Study of Parasitic Toolchain Attacks on the MCP Ecosystem. CoRR abs/2509.06572 (2025). arXiv:2509.06572 doi:10.48550/ARXIV.2509.06572
- [66] WeiBo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. 2025. When MCP Servers Attack: Taxonomy, Feasibility, and Mitigation. CoRR abs/2509.24272 (2025). arXiv:2509.24272 doi:10.48550/ARXIV.2509.24272